Building Trustworthy Federated Learning Models Using Privacy Enhancing Technologies

Project Overview

This project implements a **Trustworthy Federated Learning (FL) framework** enhanced with **Privacy Enhancing Technologies (PETs)**. The system enables multiple clients to collaboratively train a machine learning model **without sharing raw data**, while ensuring **privacy, security, and trustworthiness**.

Key technologies used: - Federated Learning using **Flower** - Deep Learning using **PyTorch** - **Differential Privacy** using **Opacus** - Result visualization using **Matplotlib**

Key Features

- Privacy-preserving federated training (no raw data sharing)
- Differential Privacy
- Trustworthy global aggregation
- Clear server and client-side logs
- Automatic generation of result graphs

Project Structure

```
trustworthy-fl/
│
├── server.py               # Federated server
├── client.py               # Federated client with DP
├── model.py                # CNN model definition
├── data.py                 # Dataset loading utilities
├── utils.py                # Metrics logging and graph generation
├── requirements.txt        # Project dependencies
├── README.md               # Project documentation
│
├── results/
│   ├── accuracy.png        # Accuracy vs rounds graph
│   ├── loss.png            # Loss vs rounds graph
│   └── metrics.csv         # Logged metrics
│
└── venv/                   # Virtual environment (optional)
```

System Requirements

- Python **3.8 or higher**
- Windows
- Minimum 8 GB RAM recommended

Setup Instructions

1 Create Virtual Environment (Recommended)

```
python -m venv venv
```

2 Activate Virtual Environment

Windows (PowerShell):

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
venv\Scripts\activate
```

3 Install Project Requirements

```
pip install -r requirements.txt
```

How to Run the Project

Step 1: Start the Federated Server

Open **Terminal 1**:

```
python server.py
```

Expected output:

```
[SERVER] Round 1 | Loss: ... | Accuracy: ...
[SERVER] Round 2 | Loss: ... | Accuracy: ...
```

Step 2: Start Federated Clients

Open **Terminal 2 (and more terminals for multiple clients)**:

```
python client.py
```

Expected output:

```
Client Training: 100%|████████|
[CLIENT] Training done | ε = 4.21
```

You can start **multiple clients** by running `client.py` in separate terminals.

Output and Results

After training completes, the following outputs are generated automatically:

Console Logs

- Federated round number
- Global loss and accuracy
- Client-side privacy budget (epsilon)

Generated Files

```
results/
├── accuracy.png   # Accuracy vs Federated Rounds
├── loss.png       # Loss vs Federated Rounds
└── metrics.csv    # Logged metrics per round
```

These graphs can be **directly included in the final project report** as proof of execution and performance.

Privacy and Trustworthiness

- **Differential Privacy:** Ensures client data confidentiality using DP-SGD
- **Privacy Budget (ε):** Displayed after each client training round
- **Trustworthy Aggregation:** Prevents direct access to client data

---

Dataset Used

- **MNIST Handwritten Digits Dataset**
- Dataset is automatically downloaded during first execution
- Data remains local to each client

Applications

- Healthcare data analysis
- Financial systems
- Distributed IoT learning
- Privacy-sensitive AI systems

Conclusion

This project successfully implements a **Trustworthy Federated Learning framework** that balances model performance with strong privacy guarantees. By integrating Differential Privacy and federated training, the system ensures secure and collaborative machine learning suitable for real-world and academic applications.